

Week 4: More ML and Model Optimization



MAISI



Building Interpretable AI in Healthcare
Michigan Data Science Team - Winter 2026

Week 4 Agenda

01.

Icebreaker!

Every great project work session starts off with a great icebreaker.

02.

Cross-Validation & Regularization

How can we be certain that our model generalizes well to unseen data?

03.

More ML Algorithms

What are some other machine learning algorithms that can be beneficial to making our predictions?

04.

Probability Calibration

How can we get accurate, interpretable probabilities for classification predictions?

01 Fun Icebreaker!!

Get to know us and start to learn about
each other with a fun icebreaker!



Icebreaker – Week 4

Share with the people around you!! :)

What are your thoughts on the following philosophical questions?

- If a tree falls in a forest when no one is around to hear it, does it even make a sound?
- If a book is written in a language that no one can understand, can it contain knowledge?
- If you constantly replace the parts of a ship with new parts, at what point is it no longer the same ship?
- What is the sound of one hand clapping?
- Can an idea exist if no one ever thinks it?





02

Cross Validation and Regularization

How can we be certain that our model generalizes well to unseen data, and how can we prevent overfitting?



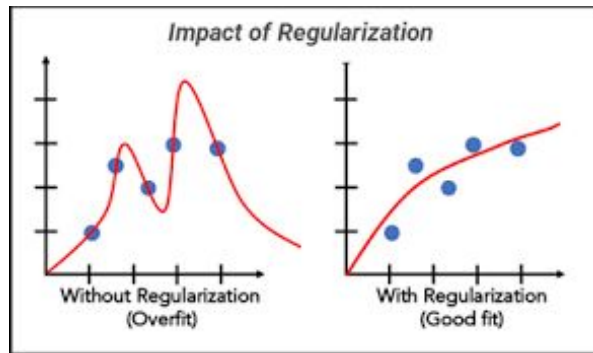
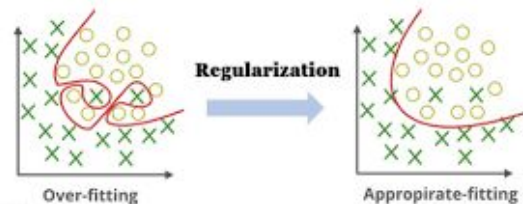
What is Cross Validation?

- **Cross Validation** is a resampling technique used in machine learning to evaluate the performance of a machine learning model
- This is achieved by splitting a dataset into k “folds,” which are randomized and equally-sized subsets of the data, and training the model on $k - 1$ of them to predict the outcome in the case of the remaining fold (called the validation set)
- This process is typically run k times, switching off which fold is the validation set each time. Model performance can then be evaluated using R^2 or $(TP+TN)/(TP+TN+FP+FN)$

What is Regularization?

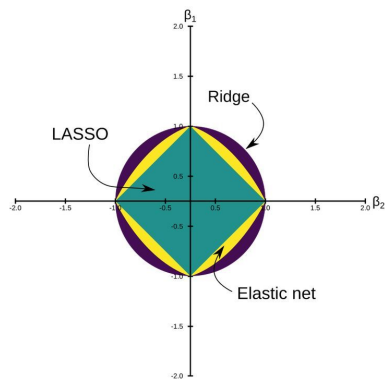
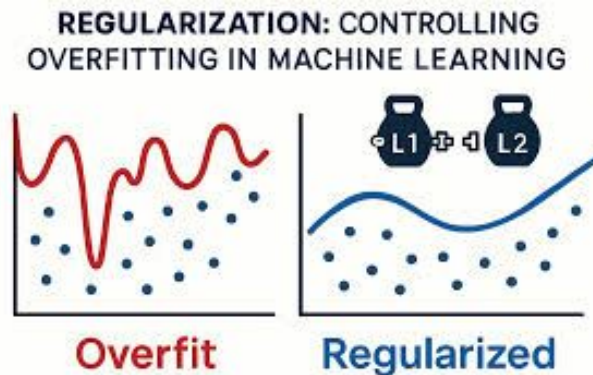
- **Regularization** is a technique used to prevent overfitting in an ML model by penalizing complexity and large weights on particular features.
 - Overfitting: Memorizing a practice test rather than learning the concepts.
- Modifies the **loss function** of the model by adding a penalty term.
 - Loss function: mathematical model quantifying the error between predicted and actual values.

What is Regularization in Machine Learning?



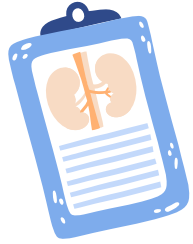
What are the Main Kinds of Regularization?

- **L1 (Lasso):** Adds the absolute value of the coefficients to the cost function
- **L2 (Ridge):** Adds the squared magnitude of all of the coefficients to the cost function.
- **ElasticNet (L1 + L2):** Combines both L1 and L2 regularization to the “cost function” of the model. In this way, you get the “best of both worlds.”



When Should I Use Each Method?

- **L1 (Lasso):** Good for selecting quality features and simplifying the coefficients of the features in your dataset.
- **L2 (Ridge):** Ideal to handle multicollinearity issues (multiple variables having a high correlation, skewing the influence of that feature on the target).
- **ElasticNet (L1 + L2):** Works for almost every case in which you would use Lasso or Ridge, though it is not advisable to use if you have an exceedingly small number of observations or if you need a very simple model that can be calculated more easily.



03

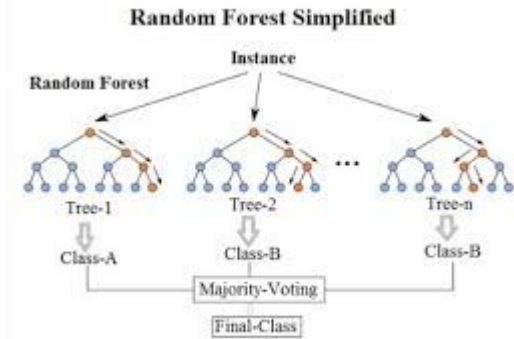
More ML Algorithms

What are a couple of other ML algorithms we can use to make predictions?



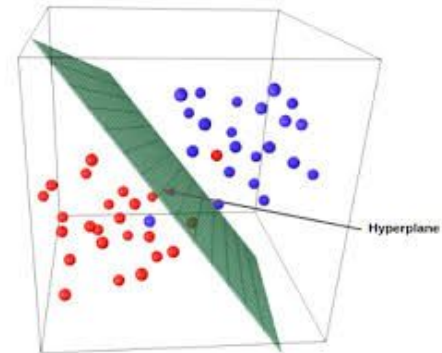
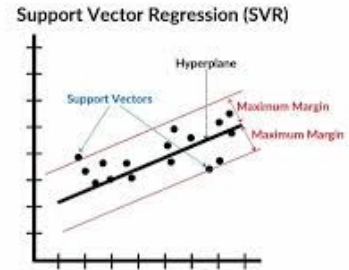
ML Algorithms - Random Forest

- Aggregates decisions made over multiple different tree and averages them out to find the most likely classification
- Made up of numerous different “trees” that are trained on different subsets of the data, sampled with replacement
- Works on both classification and regression tasks
 - Classification: takes the **mode** (most common)
 - Regression: takes the average (Σ/n)
- Highly versatile and typically very accurate predictions!



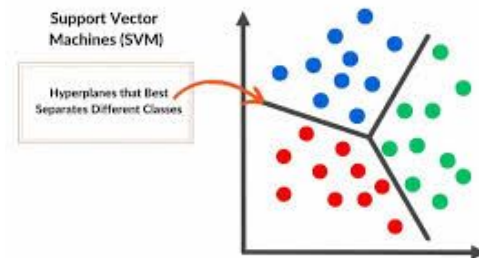
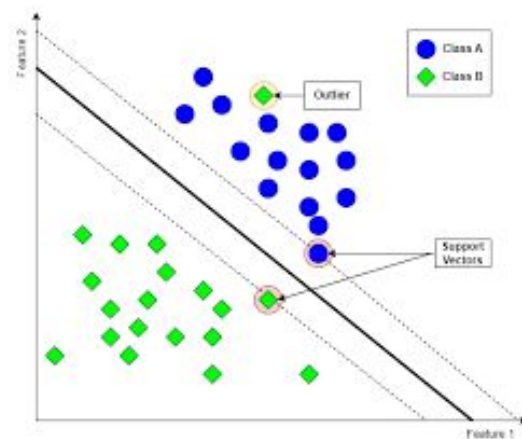
ML Algorithms – Support Vector Regressor

- **Support Vector Regressor (SVR)** is used mainly to predict continuous numerical values by attempting to fit a *hyperplane* with a defined tolerance margin around data
 - Goal: get as many data points inside the margin as possible while minimizing error from the hyperplane.
- Basically ignores points that don't fall on the hyperplane but within the tolerance margin in the error calculations.
- Often used in contexts like stock market prediction, forecasting (weather, healthcare outcomes), and other regression tasks where complex, non-linear relationships between features exist.



ML Algorithms - Support Vector Machine

- **Support Vector Machine (SVM)** is very similar to Support Vector Regressor, but used in the context of *classification* (which encompasses all of your projects)
- Works by fitting hyperplane(s) that best separate classes in the dataset, often incorporating a tolerance margin like in the last problem when calculating error.
- Very effective in high-dimensional datasets and accomplishes the same goal as SVR in reducing overfitting.
 - However, SVM has the same issues of being unsuitable for data with high multicollinearity.





Probability Calibration

How can we get accurate, interpretable probabilities for classification predictions?



What is Probability Calibration?

- **Probability Calibration** quantifies uncertainty by helping our models interpret data rationally when absolute confidence is not possible.
 - Allows the model to spit out its “belief” in an outcome rather than a prediction with a handful of summary statistics as to how it got to that conclusion
- A lot of machine learning models, as a consequence of the origin of the field, are founded on top of core statistical principles, including quantifying uncertainty.
 - E.g. AI chatbots as complex probability models, Naive Bayes, logistic regression
- This information on how likely the model claims its prediction is to be true, we can tune parameters and adjust weights to better optimize our models to make the best conclusions possible.



Why is Probability Calibration Important?

- Important to note: the proportion of observations that fit into each classification category **do not** reflect the empirical (real-world) prevalence of these categories.
 - With *probability calibration*, we can map these values to interpretable, accurate metrics
- This step is crucial in the model optimization process to make sure that the likelihoods our model returns is accurate to the real-world reality.
- Ex. A model predicts a category with 70% confidence, and that category has about a 70% chance of belonging to that category in reality.
- This is an improvement over straight-up binary classification, especially in a context like healthcare where the likelihood of an event is just as important as the diagnosis.



05

References

Here are a handful of useful guides to high-utility functions, Git demos, and Colab explanations!



Our Demo! (for reference)



Link:

<https://mdst-ai-in-healthcare.streamlit.app/>

Project Timeline

Week 1 (01/25): Icebreaker/EDA intro, choosing a good dataset

Week 2 (02/01): Data Visualization, Further EDA

Week 3 (02/08): Feature Engineering and Intro to Machine Learning

Week 4 (02/15): Machine Learning and Model Optimization

Week 5 (02/22): Conformal Predictions, Risk Control, and Multi-Class Calibration

NO MEETING 03/01 OR 03/08 – SPRING BREAK

Week 6 (03/15): SHAP Values and Feature Importance Interpretation

Week 7 (03/22): Intro to Streamlit and Project Work Time

Weeks 8 + 9 (03/29 + 04/05): Final Project work time + Data Science Night prep!



Important Functions to Know (pt. 1)

Syntax	Description
<code>import pandas as pd</code>	Import the pandas library, using the alias pd for convenience
<code>import numpy as np</code>	Import the NumPy library, using alias np for numerical operations
<code>import seaborn as sns</code>	Import Seaborn for statistical data visualization
<code>df = pd.read_csv('file.csv')</code>	Load a CSV file into a pandas DataFrame for data manipulation
<code>df.head()</code>	Returns the first 5 rows of the DataFrame, useful for quickly inspecting data
<code>df.info()</code>	Provides a concise summary of the DataFrame, including data types and non-null values

Important Functions to Know (pt. 2)

Syntax	Description
<code>df.describe()</code>	Generates descriptive statistics of numerical columns (mean, median, quartiles, etc.)
<code>df['column_name']</code>	Access a specific column in the DataFrame, works like a key in a dictionary
<code>df.drop(columns=['col'...])</code>	Drops specified columns from the DataFrame, use <code>inplace=True</code> to modify the original DataFrame
<code>df.index</code>	Returns the index labels of the DataFrame
<code>df.isnull()</code>	returns a DataFrame of boolean values , where each entry indicates whether the corresponding value in df is NaN (missing)

Important Functions to Know (pt. 3)

Syntax	Description
<code>df['column'].value_counts()</code>	Returns the count of unique values in a specific column
<code>df['column'].mean()</code>	Returns the mean value of a numerical column
<code>df.corr()</code>	Computes correlation for numerical columns to understand relationships
<code>df.columns</code>	Lists all column names in the DataFrame , useful for renaming or viewing dataset structure
<code>df.shape</code>	Returns # of rows and columns

Important Functions to Know (pt. 4)

Syntax	Description
<code>df.rename(columns={'old' : 'new'}, inplace=True)</code>	Renames specific columns to new names
<code>df.groupby('category')['value']</code>	Groups the DataFrame using a specified column to perform aggregate functions (e.g., <code>.sum()</code> , <code>.mean()</code>)
<code>df['new_column'] = df['column'].apply(function)</code>	Applies a function to each element or column/row
<code>df.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the DataFrame
<code>df.dropna()</code>	Removes rows or columns containing missing values

Important Functions to Know (pt. 5)

Syntax	Description
<code>df.to_numpy()</code>	Converts Pandas DataFrame into a NumPy array
<code>np.shape</code>	Returns a tuple representing the dimensions (rows, columns) of the array (similar to <code>df.shape</code>)
<code>np.reshape()</code>	Reshapes an original numpy array to the specified dimensions (rows, columns)
<code>np.array()</code>	Creates an N-dimensional array (ndarray) which is more memory efficient than standard python lists
<code>np.unique()</code>	Returns the unique elements in a NumPy array (also returns the counts of each element given the keyword argument, <code>return_counts=True</code>)

Important Functions to Know (pt. 6)

Syntax	Description
<code>pd.cut()</code>	Turns continuous numerical data into categorical data
<code>pd.get_dummies(drop_first = True)</code>	Dummy encodes categorical data (used with <code>drop_first</code> argument)
<code>OrdinalEncoder()</code>	Function for encoding categorical information that has an order
<code>train_test_split(X, y, test_size=0.2, random_state=42)</code>	Splits data into a training set and a testing set . The test set size can be changed
<code>LinearRegression()</code>	Used for initializing a linear regression model
<code>LogisticRegression()</code>	Used for initializing a logistic regression model

Important Functions to Know (pt. 7)

Syntax	Description
StandardScaler()	Turns continuous numerical data into categorical data
model.fit(X_train, y_train)	Fits a model to the training data
model.predict(X_test)	Uses the test data to make predictions
.coef_	Attribute for getting the models coefficients (for linear and logistic regression)
mean_squared_error()	Calculates the average squared distance between the predicted and actual data points
mean_absolute_error()	Calculates the average distance between the predicted and actual values

How to Upload Files into Colab

Click on the folder in the sidebar



The screenshot shows a Google Colab notebook interface. On the left, the 'Files' sidebar is open, showing a folder named 'sample_data'. A red arrow points to this folder from the text 'Click on the folder in the sidebar'. The notebook content area has a title 'Week 1 - Pandas Practice'. Below the title, there is a text prompt: 'Here is where you import the libraries necessary to perform the following tasks!'. The first code cell contains the following Python code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Below the code cell, a message indicates 'Mounted at /content/drive'. The next text prompt says 'Load the Google Forms .csv into a Pandas dataframe.'. The second code cell contains:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

The third text prompt says 'Print out the .head() and the datatypes.'. The third code cell contains:

```
df.head()
```

Below the code cell, the output of the `df.head()` command is displayed as a table with 10 columns. The columns are: 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If not, leave blank. (Answer in state code)', 'If you're American, what country are you from? If American,', '(Approximately) How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', 'How many roommates do you have?', and 'How many football games have you played? (Approximate are not)'. The first row of data is partially visible.

At the bottom of the interface, there are tabs for 'Variables' and 'Terminal', and a status bar indicating 'Python 3'.

How to Upload Files into Colab

Click the upload button and select the file you want to upload



The image shows the Google Colab interface. On the left, the 'Files' pane shows a directory structure with a folder named 'sample_data' and a file named 'MDST Week 1 - Pandas Practice.csv'. A red arrow points from the text instruction to the upload button (a square with an upward arrow) in the file manager. The main code area is titled 'Week 1 - Pandas Practice' and contains the following code:

```
import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Below the code, it says 'Mounted at /content/drive'. The next code block is:

```
df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

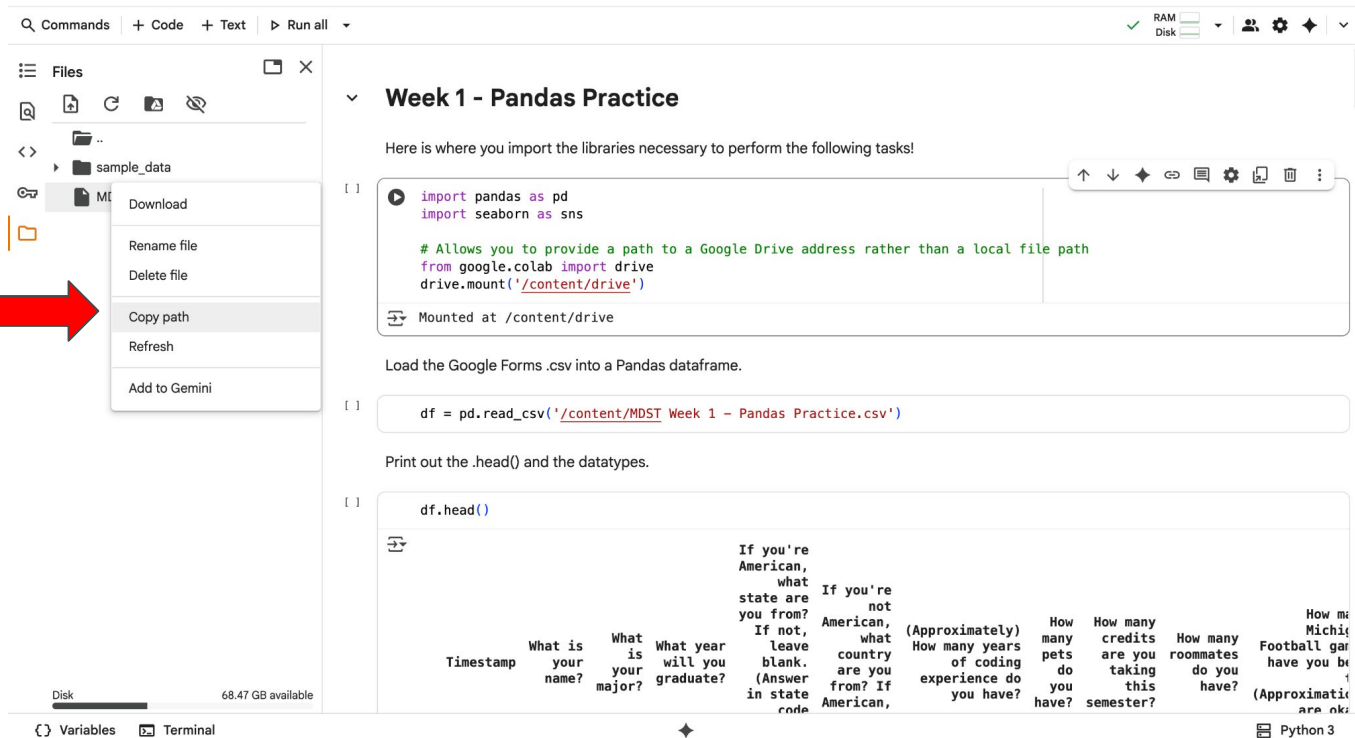
Below that, it says 'Print out the .head() and the datatypes.' The final code block is:

```
df.head()
```

The output of the code is a preview of the CSV data, showing columns like 'Timestamp', 'What is your name?', 'What is your major?', 'What year will you graduate?', 'If you're American, what state are you from?', 'If not, leave blank. (Answer in state code)', 'If you're not American, what country are you from? If American, (Approximately) How many years of coding experience do you have?', 'How many pets do you have?', 'How many credits are you taking this semester?', 'How many roommates do you have?', and 'How many Michigan Football games have you been to? (Approximately)'. The interface also shows a 'Disk' usage bar at the bottom left indicating 68.47 GB available, and a 'Terminal' tab at the bottom.

How to Upload Files into Colab

Click the three dots and copy the path. Put this in your read function



The screenshot shows the Google Colab interface. On the left, the 'Files' pane displays a folder named 'sample_data' containing a file 'MDST Week 1 - Pandas Practice.csv'. A context menu is open for this file, with the 'Copy path' option highlighted. A red arrow points from the text 'Click the three dots and copy the path. Put this in your read function' to this option. The main editor area is titled 'Week 1 - Pandas Practice' and contains the following text and code:

Here is where you import the libraries necessary to perform the following tasks!

```
[ ] import pandas as pd
import seaborn as sns

# Allows you to provide a path to a Google Drive address rather than a local file path
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Load the Google Forms .csv into a Pandas dataframe.

```
[ ] df = pd.read_csv('/content/MDST Week 1 - Pandas Practice.csv')
```

Print out the .head() and the datatypes.

```
[ ] df.head()
```

The output of `df.head()` is a table with columns: Timestamp, What is your name?, What is your major?, What year will you graduate?, If you're American, what state are you from?, If not, leave blank. (Answer in state code), If you're American, what country are you from? If American, (Approximately) How many years of coding experience do you have?, How many pets do you have?, How many credits are you taking this semester?, How many roommates do you have?, How many Football games have you been to? (Approximately) How many are ok.

At the bottom, the 'Variables' pane shows 'df' and the 'Terminal' pane shows 'Python 3'.

How to Mount Your Google Drive

```
from google.colab import drive

drive.mount('/content/drive')

pd.read_csv('/content/drive/MyDrive/[FILE NAME]')
```

****NOTE:** If you saved your dataset in a folder (not just loose in your MyDrive folder), you will need to add further code to the file path before the file name in the final line. For example, if I saved my data called “alzheimers.csv” in a folder called “MDST-W26,” my read_csv statement would read:

```
pd.read_csv('/content/drive/MyDrive/MDST-W26/alzheimers.csv')
```

How to Create a GitHub Repository (yay for version control!)

- I. Navigate to GitHub, click your profile picture, and select “Repositories.” Then, click the green “New” button next to the “Language” dropdown menu

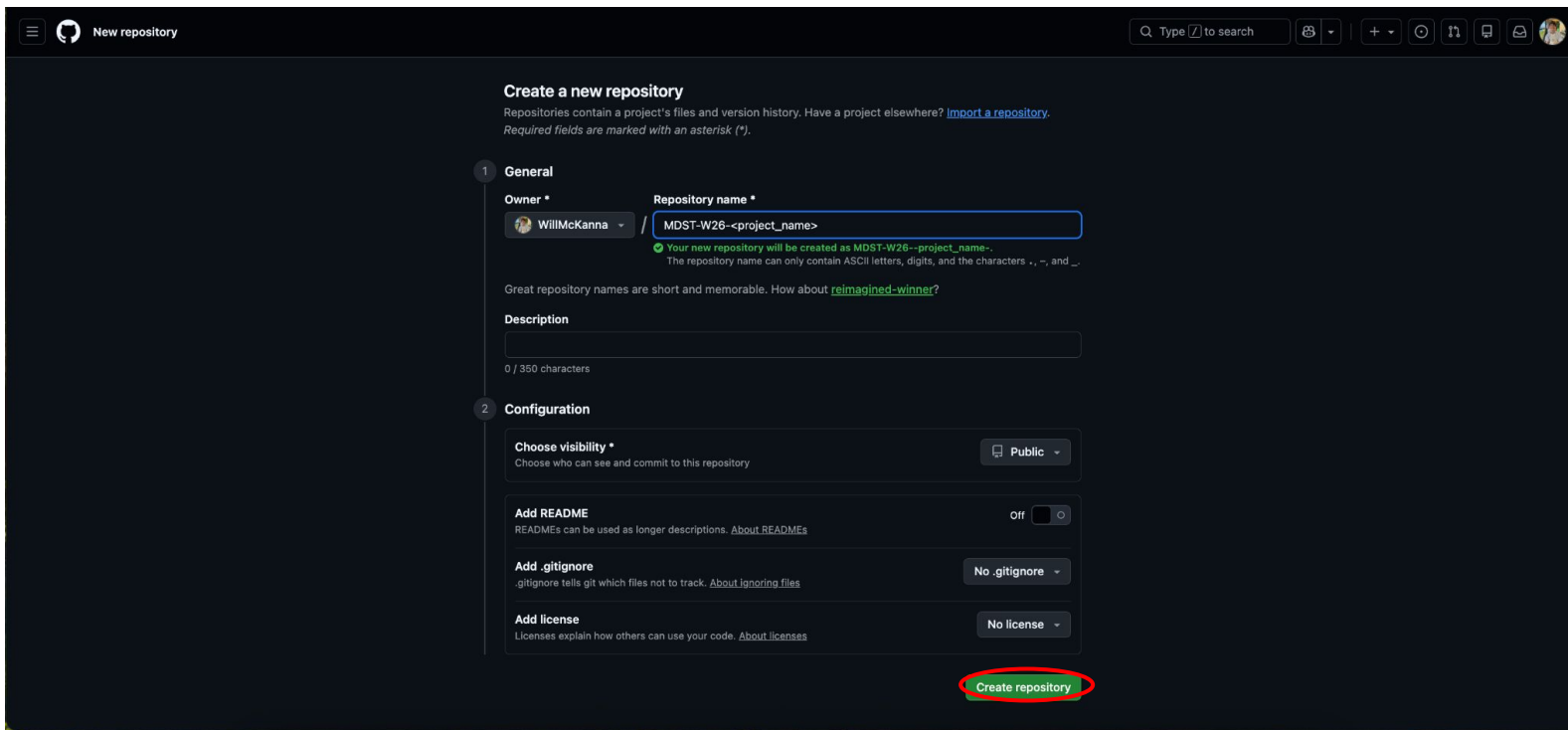
The screenshot shows the GitHub profile page for user WillMcKanna. The 'Repositories' tab is selected in the top navigation bar and the right sidebar. The sidebar menu on the right has 'Repositories' circled in red. The main content area displays a list of repositories:

- eeecs281-proj1** (Private): Personal repository for version control of EECS 281 - Project 1 at the University of Michigan in the Winter 2026 semester. C++ · Updated 16 hours ago.
- F25-criminal-risk-analysis** (Public): Michigan Data Science Team F25. Jupyter Notebook · Updated on Nov 25, 2025.
- eeecs280-proj3** (Private): Project 3 for EECS280 Study Abroad. C++ · Updated on Jul 22, 2025.
- eeecs280-proj2** (Private): (partially visible)

The user's profile picture shows a young man with dark hair wearing a light-colored apron. The bottom of the page shows the URL: <https://github.com/WillMcKanna?tab=repositories>.

How to Create a GitHub Repository (yay for version control!)

2. Name your repository something memorable, then click “Create Repository”



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * WillMcKanna / **Repository name *** MDST-W26-<project_name>

✔ Your new repository will be created as MDST-W26-<project_name>-.
The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [reimagined-winner](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository. Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#). Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#). No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#). No license

[Create repository](#)

How to Create a GitHub Repository (yay for version control!)

3. Then, populate your repo either with your files, OR click “creating a new file”

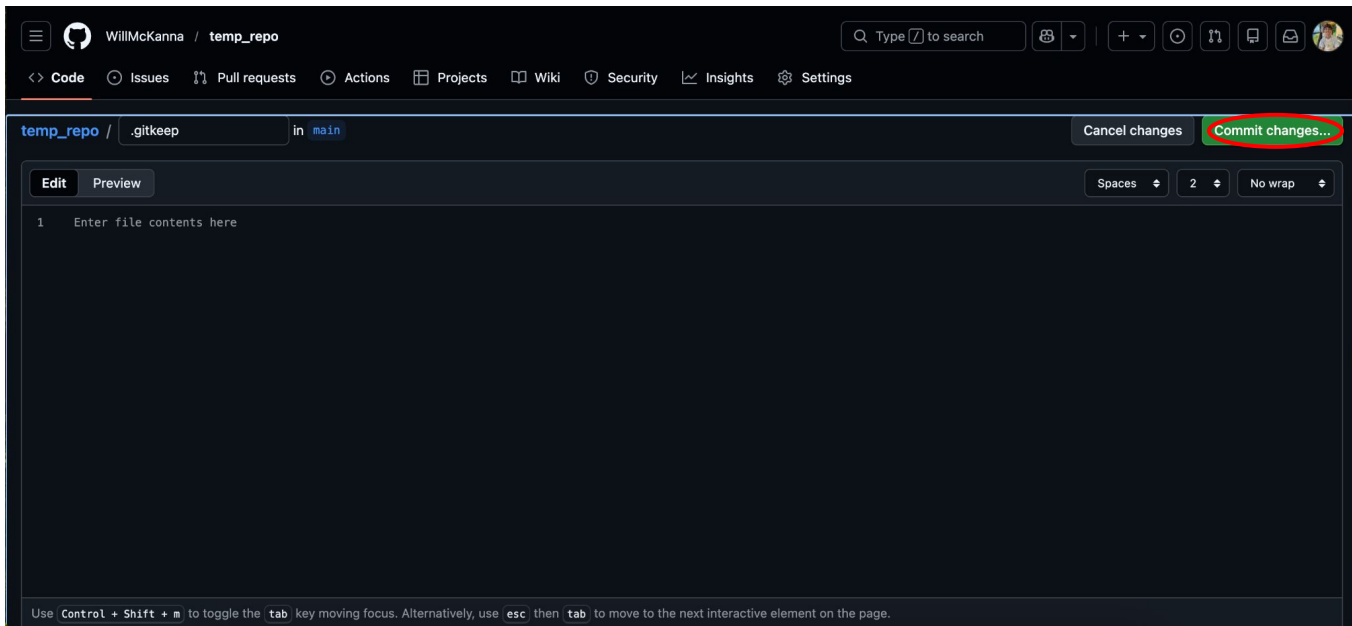
The screenshot shows the GitHub interface for a repository named 'temp_repo' by user 'WillMcKanna'. The repository is public. The page offers several options for getting started:

- Start coding with Codespaces:** A section with a laptop icon and text: "Add a README file and start coding in a secure, configurable, and dedicated development environment." Below this is a button labeled "Create a codespace".
- Add collaborators to this repository:** A section with a person icon and text: "Search for people using their GitHub username or email address." Below this is a button labeled "Invite collaborators".
- Quick setup — if you've done this kind of thing before:** A section with a dark blue background. It shows three options: "Set up in Desktop", "HTTPS", and "SSH". The "SSH" option is selected, and the URL "https://github.com/WillMcKanna/temp_repo.git" is displayed. Below this, it says "Get started by creating a new file or uploading an existing file. We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#)." The phrase "creating a new file or uploading an existing file." is circled in red.
- ...or create a new repository on the command line:** A section with a dark blue background showing the following commands:

```
echo "# temp_repo" >> README.md
git init
git add README.md
```

How to Create a GitHub Repository (yay for version control!)

3(b). If you click “create a new file,” entitle it “.gitkeep” and click “Commit changes”



How to Create a GitHub Repository (yay for version control!)

4. Your repo is made! Continue adding to it by clicking the “Add file” dropdown menu

The screenshot shows the GitHub interface for a repository named 'temp_repo' by user 'WillMcKanna'. The repository is public. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the repository name, there are buttons for Pin, Watch (0), Fork (0), and Star (0). The main content area shows the 'main' branch with 1 branch and 0 tags. A search bar 'Go to file' is present. The 'Add file' dropdown menu is circled in red. Below this, there is a section for the '.gitkeep' file, created by WillMcKanna, with a commit hash 'a168bb7' and a timestamp 'now'. The 'README' section is also visible, with a button to 'Add a README'. The right sidebar contains sections for 'About' (No description, website, or topics provided), 'Activity' (0 stars, 0 watching, 0 forks), 'Releases' (No releases published, Create a new release), and 'Packages' (No packages published, Publish your first package).